Package 'facereaderconverter'

September 11, 2026

Title Process of FaceReader outputs and emotional episodes

Version 0.3.2

Description Tools to convert FaceReader outputs; Markup of emotional episodes.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

Depends R (>= 4.1.0)

Imports dplyr,
hms,
janitor,
readr,
readxl,
stringr,
tidyselect,
tools,
tibble,
Rcpp,
data.table,
tidyr

LinkingTo Rcpp

Suggests openxlsx,
testthat (>= 3.0.0)

Config/testthat/edition 3

Config/roxygen2/version 8.1.0

Contents

`add_delta_column` *Add delta column to coding_df*

Description

`add_delta_column()` is deprecated because `convert_to_episodes()` now computes and returns `delta` automatically.

Usage

```
add_delta_column(
  coding,
  delta = 0.1,
  delta_window = 0.2,
  fps = 30L,
  cores = 0L
)
```

Arguments

`coding`	Data frame with columns: id, subject, emotion, frame (or video_time), value, or an `fr_coding` object.
`delta`	Threshold for both up and down.
`delta_window`	Window size in seconds.
`fps`	Frames per second.
`cores`	integer Number of threads to use. Default 0 is auto.

Value

coding_df with a `delta` column where 1 means up and 0 means down.

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
add_delta_column(
    coding_df,
    delta_window = 0.2,
    delta = 0.1,
    fps = 30
```

```
 )

## End(Not run)
```

`convertFRDirectory` *Convert a directory of Facereader TXT to CSV*

Description

Reads all Txt files in a folder and sends them through convertFRFiles.

Usage

```
convertFRDirectory(
  inpath,
  outpath = inpath,
  recursive = TRUE,
  pattern = NULL,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  save_metadata = outpath,
  metadata_filename = "metadata.csv",
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  cores = 0L,
  ...
)
```

Arguments

`inpath`	Path to an existing .txt file.
`outpath`	Path to save the csvs to defaults to the inpath
`recursive`	Bool as to whether to look for all files in directory (`TRUE`) or just the root folder (`FALSE`)
`pattern`	a regex pattern of files to test, if `NULL` then will look for all txt files
`values_as_numeric`	Save values as numeric, where applicable
`clean_names`	returns janitor-style clean names
`save_metadata`	save the metadata as a csv in the outpath, set to NULL to not save
`metadata_filename`	filename of the metadata csv
`fail_codes`	adds a column with the fail reason, True or False. Column then has 0 for success, 1 for fit_failed, 2 for find_failed
`duplicate_timecodes_as_error`	throws an error if there are duplicate timecodes, if FALSE then throws warning
`cores`	integer Number of threads to use. Default 0 is auto.
`...`	arguments passed as necessary

Value

Invisibly returns the metadata.

Examples

```
## Not run:
convertFRDirectory(
  inpath="directory_of_txt_files",
  outpath="directory_to_save_csvs_to",
  values_as_numeric = TRUE
)

## End(Not run)
```

---

convertFRExcelFiles *Load FaceReader Excel exports*

Description

Reads a FaceReader `.xlsx` file, detects the header row automatically, and returns the parsed data frame. The function follows the same post-processing conventions as `convertFRFiles()` where applicable.

Usage

```
convertFRExcelFiles(
  inpath,
  return_data = TRUE,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  sheet = 1,
  ...
)
```

Arguments

`inpath`	Path to an existing `.xlsx` file.
`return_data`	Return the parsed data instead of metadata.
`values_as_numeric`	Convert `Video Time` to hms and detailed values to numeric.
`clean_names`	Apply `janitor::clean_names()` to the data.
`fail_codes`	Add a `fail_code` column for detailed exports.
`duplicate_timecodes_as_error`	Throw an error if duplicate timecodes are found.
`sheet`	Excel sheet to read. Defaults to the first sheet.
`...`	Additional arguments passed to `janitor::clean_names()`.

Value

Invisibly returns the parsed data when `return_data = TRUE`; otherwise returns metadata about the import.

Examples

```
## Not run:
convertFRExcelFiles(
  inpath = "FaceReaderOutput.xlsx",
  return_data = TRUE,
  values_as_numeric = TRUE
)

## End(Not run)
```

---

convertFRFiles *Convert Facereader TXT to CSV*

Description

Reads a .txt file, detects the FaceReader header row automatically, guesses the delimiter from the header line, writes a .csv with the same basename in the same directory, and returns the path to the created CSV (invisibly).

Usage

```
convertFRFiles(
  inpath,
  outpath = paste0(tools::file_path_sans_ext(inpath), ".csv"),
  return_data = FALSE,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  ...
)
```

Arguments

`inpath`	Path to an existing .txt file.
`outpath`	Path to save the csv to defaults to the inpath
`return_data`	Bool to return the data from the txt rather than the metadata
`values_as_numeric`	Save values as numeric, where applicable
`clean_names`	returns janitor-style clean names
`fail_codes`	adds a column with the fail reason, True or False. Column then has 0 for success, 1 for fit_failed, 2 for find_failed
`duplicate_timecodes_as_error`	throws an error if there are duplicate timecodes, if FALSE then throws warning
`...`	arguments passed as necessary

Value

Invisibly returns the metadata.

Examples

```
## Not run:
convertFRFiles(
  inpath="FaceReaderOutput.txt",
  values_as_numeric = TRUE
)

## End(Not run)
```

---

convert_to_episodes *Convert coding_df to episodes*

Description

Convert coding_df to episodes

Usage

```
convert_to_episodes(
  coding_df,
  T_up = 0.2,
  T_down = 0.1,
  delta = 0.1,
  delta_window = 0.2,
  min_dur_sec = 0.1,
  consecutive_missing = 150L,
  fps = 30L,
  cores = 0L
)
```

Arguments

`coding_df`	a dataframe or otherwise from a FaceReader output. id and subject should be present.
`T_up`	numeric Upper threshold for entering an episode. Default: 0.2.
`T_down`	numeric Lower threshold for exiting an episode. Default: 0.1.
`delta`	numeric Threshold used when computing the returned `delta` column, as the minimum k-step change required. It no longer controls episode boundaries. Default: 0.1.
`delta_window`	numeric Time window (in seconds) used to derive `k`, the lag for the returned k-step difference `delta` column. It no longer controls episode boundaries. Default: 0.1.
`min_dur_sec`	numeric Minimum episode duration (in seconds). Default: 0.1.

`consecutive_missing`
: integer Maximum allowed consecutive missing (NA) frames while in-state before forcing episode end. Default: 150L.

`fps`
: integer Frames per second (sampling rate of the data). Default: 30L.

`cores`
: integer Number of threads to use. Default 0 is auto.

Details

Episode boundaries are determined only by `T_up` and `T_down`. The returned `delta` column is computed separately and delta-up rows are also returned in `deltas` for downstream functions such as `reaction_rate()`. The function uses the exported native binding `hysteresis_state` if available; otherwise it will error. It relies on **data.table** for fast grouping and joins.

Value

A list with four elements:

episodes data.table of detected episodes with columns `start_frame`, `end_frame`, `n_frames`, `duration_s`, `id`, `subject`, `emotion`, `start_time`, `end_time`, `run_id`, and `max_value`.

deltas data.table of delta-up reaction events with the same columns as `episodes`, except `delta_id` replaces `run_id`, and with `max_delta`, the value range from one delta window before the event start through the event end.

coding Annotated data.table containing the original columns plus `id`, `subject`, `emotion`, `value`, `delta`, `delta_id`, `run_id`, `status`, and `in_state`. `status` marks episode boundaries with `1L` at the start frame and `0L` at the end frame; `in_state` is `TRUE` for frames inside detected episodes.

metadata Metadata used to create the returned object.

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
convert_to_episodes(coding_df)

## End(Not run)
```

`delta` *Add delta column alias*

Description

`delta()` is deprecated because `convert_to_episodes()` now computes and returns `delta` automatically.

Usage

```
delta(coding, delta = 0.1, delta_window = 0.2, fps = 30L, cores = 0L)
```

Arguments

coding	Data frame with columns: id, subject, emotion, frame (or video_time), value, or an `fr_coding` object.
delta	Threshold for both up and down.
delta_window	Window size in seconds.
fps	Frames per second.
cores	integer Number of threads to use. Default 0 is auto.

Value

coding_df with a `delta` column where 1 means up and 0 means down.

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
delta(
  coding_df,
  delta_window = 0.2,
  delta = 0.1,
  fps = 30
)

## End(Not run)
```

---

delta_episodes *Add create delta episodes*

Description

Add create delta episodes

Usage

```
delta_episodes(coding, fps = 30L, cores = 0L)
```

Arguments

coding	Data frame with columns: id, subject, emotion, frame (or video_time), value, or an `fr_coding` object
fps	Frames per second
cores	integer Number of threads to use. Default 0 is auto.

Value

episodes data.table of detected episodes with columns `start_frame`, `end_frame`,`start_time`, `end_time`, `n_frames`, `duration_s`, `id`, `subject`, `emotion`, and `run_id`.

Examples

```
## Not run:
coding_df = read.csv("testdata_detailed.csv")
x=add_delta_column(
    coding_df,
    delta_window = 0.2,
    delta = 0.1,
    fps = 30
 )
delta_episodes(x)


## End(Not run)
```

---

export_shared_synchrony_clips

Export clips for shared synchronous episodes

Description

Selects shared synchronous intervals with the largest combined emotion values and exports the corresponding video segments with FFmpeg. Frame ranges are inclusive: a range from frame 0 through frame 0 has duration `1 / fps`.

Usage

```
export_shared_synchrony_clips(
  coded_data,
  shared_synchrony,
  video_paths,
  n = 10L,
  emotion = "happy",
  buffer = 0,
  buffer_units = c("seconds", "frames"),
  buffer_frames = 0L,
  buffer_seconds = 0,
  output_path = NULL,
  output = c("zip", "folder"),
  overwrite = FALSE,
  ffmpeg = "ffmpeg"
)
```

Arguments

`coded_data`	A converted `fr_coding` object with `metadata$fps`.
`shared_synchrony`	A table returned by `shared_synchronous_episodes()`.
`video_paths`	Named character vector of source video paths. Names must match values in the shared-synchrony `id` column.
`n`	Number of highest-ranked shared intervals to export.

`emotion`	Optional character vector of emotions to retain.
`buffer`	A non-negative number for a symmetric buffer, or a named two-element vector with names `before` and `after` for asymmetric buffers. Units are controlled by `buffer_units`.
`buffer_units`	Buffer units: `"seconds"` or `"frames"`.
`buffer_frames`	Non-negative whole-number symmetric frame buffer. Kept for compatibility; use `buffer` and `buffer_units` for new code.
`buffer_seconds`	Non-negative symmetric time buffer in seconds. Kept for compatibility; use `buffer` and `buffer_units` for new code.
`output_path`	Optional ZIP file path when `output = "zip"`, or output directory when `output = "folder"`. By default, output is placed beside the selected source video. All selected videos must then be in the same folder.
`output`	Output type: `"zip"` (the default) or `"folder"`.
`overwrite`	Logical; overwrite an existing ZIP archive or non-empty output folder. Defaults to `FALSE`.
`ffmpeg`	Path to the FFmpeg executable or its command name.

Value

A `data.table` manifest invisibly. For ZIP output, the manifest is also included in the archive as `manifest.csv`.

Examples

```
## Not run:
coded_data <- convert_to_episodes(coding_data)
shared <- shared_synchronous_episodes(coded_data)
export_shared_synchrony_clips(
  coded_data,
  shared,
  video_paths = c("1" = "recording.mp4"),
  n = 10,
  buffer = c(before = 2, after = 1),
  output_path = "recording-shared-clips.zip"
)

## End(Not run)
```

`loadFRfile` *Load FaceReader files into memory*

Description

Reads a FaceReader export file into memory by dispatching to the existing TXT, Excel, or CSV readers based on file extension. The wrapper always returns the parsed data and never writes an output file.

Usage

```
loadFRfile(
  inpath,
  values_as_numeric = TRUE,
  clean_names = TRUE,
  fail_codes = FALSE,
  duplicate_timecodes_as_error = TRUE,
  sheet = 1,
  ...
)
```

Arguments

`inpath`	Path to an existing FaceReader export file.
`values_as_numeric`	Convert `Video Time` to `hms` and detailed values to numeric where applicable.
`clean_names`	Apply `janitor::clean_names()` to the data.
`fail_codes`	Add a `fail_code` column for detailed exports.
`duplicate_timecodes_as_error`	Throw an error if duplicate timecodes are found.
`sheet`	Excel sheet to read when importing `.xlsx` files.
`...`	Additional arguments passed to the underlying importer.

Value

Invisibly returns the parsed data frame.

Examples

```
## Not run:
loadFRfile(
  inpath = "FaceReaderOutput.txt",
  values_as_numeric = TRUE
)

## End(Not run)
```

`locf` *Apply LOCF and Extract Episodes*

Description

Applies Last Observation Carried Forward (LOCF) logic to the `coding` data.table (from `convert_to_episodes`), imputing missing `run_id` values within blocks of non-missing `value`, grouped by `id`, `subject`, and `emotion`. Returns both the imputed coding table and a summary of contiguous episodes.

Usage

```
locf(coding, fps = 30, consecutive_missing = NULL)
```

Arguments

`coding` A datatable, dataframe or fr_coding object as returned by `convert_to_episodes`, containing columns: `id`, `subject`, `emotion`, `video_time`, `value`, and `run_id`.

`fps` Frames per second (sampling rate of the data). Default: 30L or inherited from an fr_coding object

`consecutive_missing` Maximum allowed consecutive missing (NA) frames while in-state before forcing episode end. Default: 150L or inherited from an fr_coding object

Value

A list with four elements:

- **episodes** data.table of contiguous episodes after LOCF, with columns `start_frame`, `end_frame`, `n_frames`, `duration_s`, `id`, `subject`, `emotion`, and `run_id`.
- **deltas** data.table of delta-up events, passed through unchanged when the input is an `fr_coding` object.
- **coding** Annotated data.table with LOCF-imputed `run_id`.
- **metadata** Metadata from fr_coding object.

`map_paths` *Map files from an input root to an output root, preserve structure, create dirs.*

Description

Map files from an input root to an output root, preserve structure, create dirs.

Usage

```
map_paths(input_dir, output_dir, files)
```

Arguments

`input_dir` Character scalar. The source root directory.

`output_dir` Character scalar. The destination root directory.

`files` Character vector. File paths under input_dir to be remapped.

Value

```
Character vector of output file paths (same length as files).
```

parse_time_to_frame *Parse video time to frame index*

Description

Convert a time string into a frame index given frames-per-second (fps). Supports "HH:MM:SS", "MM:SS", or plain seconds and is vectorised over `video_time`.

Usage

```
parse_time_to_frame(video_time, fps)
```

Arguments

video_time	Character or numeric. Time strings in "HH:MM:SS", "MM:SS", or numeric seconds.
fps	Numeric. Frames per second.

Value

Integer vector. Frame indices computed as round(seconds * fps).

Examples

```
parse_time_to_frame("00:01:23.5", fps = 30)
parse_time_to_frame("1:23.5", fps = 30)
parse_time_to_frame("83.5", fps = 30)
```

reaction_rate *Calculate reaction rate from converted episodes*

Description

Reaction rate mirrors `synchrony()` but uses the numerator subject's delta-up events as the reaction signal. Denominator episodes come from `convert_to_episodes()`, and a reaction is counted when the numerator subject has at least one delta event within the constrained denominator episode window.

Usage

```
reaction_rate(
  coded_data,
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  exclude_start = 0.1,
  exclude_start_frames = NULL,
  fps = 30L,
  subject_names = NULL,
  exclude_emotions = "neutral",
  minimum_threshold = 0
)
```

Arguments

- `coded_data` — Output from `convert_to_episodes()`, which includes frame-level `delta` values in `coded_data$coding` and a reaction-event table in `coded_data$deltas`.
- `time_limit` — Maximum episode-window length in seconds when a frame constraint is applied.
- `time_limit_frames` — Optional maximum episode-window length in frames. If supplied, this takes precedence over `time_limit`.
- `constraint_method` — Character scalar controlling how the reaction window ends. `"strict"` uses the earlier of the observed episode end and the requested time limit. `"episode"` uses the observed episode end and ignores `time_limit`. `"loose"` uses the later of the observed episode end and the requested time limit. `"frames"` uses only the requested time limit and ignores the observed episode end. Default is `"episode"`.
- `exclude_start` — Minimum delay, in seconds from the denominator episode start, before numerator `delta == 1` values count as a reaction.
- `exclude_start_frames` — Optional minimum delay in frames. If supplied, this takes precedence over `exclude_start`.
- `fps` — Frames per second used to convert between seconds and frames when `coded_data$metadata$fps` is unavailable.
- `subject_names` — Character vector of subject labels to compare. If `NULL`, all unique subjects in `coded_data$coding` are used.
- `exclude_emotions` — Character vector of emotions to exclude from the denominator calculation. Default is `"neutral"`.
- `minimum_threshold` — Numeric scalar in `[0, 1]`. Denominator episodes are kept only when at least this proportion of numerator frames in the constrained window have non-missing `values`. Default is `0`.

Value

A data.table with columns `id`, `denominator`, `numerator`, `emotion`, `n_episodes`, `n_reactions`, and `reaction_rate`.

See Also

`reaction_rate_by_episode()`

Examples

```
## Not run:
coded_data <- convert_to_episodes(coding_df)
reaction_rate(coded_data)

## End(Not run)
```

`reaction_rate_by_episode` *Calculate reaction rate by denominator episode*

Description

Calculate reaction rate by denominator episode

Usage

```
reaction_rate_by_episode(
  coded_data,
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  exclude_start = 0.1,
  exclude_start_frames = NULL,
  fps = 30L,
  subject_names = NULL,
  exclude_emotions = "neutral",
  minimum_threshold = 0
)
```

Arguments

- `coded_data` Output from `convert_to_episodes()`, which includes frame-level `delta` values in `coded_data$coding` and a reaction-event table in `coded_data$deltas`.
- `time_limit` Maximum episode-window length in seconds when a frame constraint is applied.
- `time_limit_frames` Optional maximum episode-window length in frames. If supplied, this takes precedence over `time_limit`.
- `constraint_method` Character scalar controlling how the reaction window ends. `"strict"` uses the earlier of the observed episode end and the requested time limit. `"episode"` uses the observed episode end and ignores `time_limit`. `"loose"` uses the later of the observed episode end and the requested time limit. `"frames"` uses only the requested time limit and ignores the observed episode end. Default is `"episode"`.
- `exclude_start` Minimum delay, in seconds from the denominator episode start, before numerator `delta` == 1 values count as a reaction.
- `exclude_start_frames` Optional minimum delay in frames. If supplied, this takes precedence over `exclude_start`.
- `fps` Frames per second used to convert between seconds and frames when `coded_data$metadata$fps` is unavailable.
- `subject_names` Character vector of subject labels to compare. If `NULL`, all unique subjects in `coded_data$coding` are used.
- `exclude_emotions` Character vector of emotions to exclude from the denominator calculation. Default is `"neutral"`.

`minimum_threshold`
: Numeric scalar in `[0, 1]`. Denominator episodes are kept only when at least this proportion of numerator frames in the constrained window have non-missing `values`. Default is `0`.

Value

A data.table with columns `id`, `denominator`, `numerator`, `emotion`, `run_id`, `start_frame`, `end_frame`, `n_frames`, `present_prop`, and `reaction`.

See Also

`reaction_rate()`

Examples

```
## Not run:
coded_data <- convert_to_episodes(coding_df)
reaction_rate_by_episode(coded_data)

## End(Not run)
```

`shared_synchronous_episodes` *Map shared synchronous episodes*

Description

Maps pairs of same-emotion episodes that overlap while both subjects are in state. Each unordered pair of source episodes is returned once. A source episode may therefore occur in multiple rows when it overlaps more than one episode of the other subject.

Usage

```
shared_synchronous_episodes(
  coded_data,
  subject = "subject",
  id = "id",
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  fps = 30L,
  missing_threshold = 0,
  exclude_emotions = "neutral"
)
```

Arguments

`coded_data`
: Output from `convert_to_episodes()`.

`subject`
: Character scalar giving the column in `coded_data$coding` and `coded_data$episodes` that identifies the subject. Default is `"subject"`.

id
: Character scalar giving the column in `coded_data$coding` and `coded_data$episodes` that identifies the case or dyad. Default is `"id"`.

time_limit
: Maximum episode-window length in seconds. `NULL` is allowed when `time_limit_frames` is supplied or when `constraint_method` is `"episode"`. Default is `3`.

time_limit_frames
: Optional maximum episode-window length in frames. If supplied, this takes precedence over `time_limit`.

constraint_method
: Character scalar controlling how the episode window ends: `"strict"`, `"episode"`, `"loose"`, or `"frames"`. Default is `"episode"`.

fps
: Frames per second used to convert `time_limit` to frames. Default is `30`.

missing_threshold
: Numeric scalar in `[0, 1]`. Denominator and numerator episodes are kept only when the comparison subject is present for at least this proportion of frames within the episode. Default is `0`.

exclude_emotions
: Character vector of emotions to exclude from the denominator calculation. Default is `"neutral"`.

Value

A data.table with episode-pair identifiers, their inclusive shared frame interval, each source episode's maximum value, the larger maximum, and their combined value.

See Also

`synchrony_by_episode()`

Examples

```
## Not run:
coded_data <- convert_to_episodes(coding_data)
shared_synchronous_episodes(coded_data)

## End(Not run)
```

`synchrony` *Calculate synchrony from converted episodes*

Description

Calculate synchrony from converted episodes

Usage

```
synchrony(
  coded_data,
  subject = "subject",
  id = "id",
  time_limit = 3,
  time_limit_frames = NULL,
```

```
  constraint_method = "episode",
  fps = 30L,
  missing_threshold = 0,
  exclude_emotions = "neutral"
)
```

Arguments

- `coded_data` — Output from `convert_to_episodes()`.
- `subject` — Character scalar giving the column in `coded_data$coding` and `coded_data$episodes` that identifies the subject. Default is `"subject"`.
- `id` — Character scalar giving the column in `coded_data$coding` and `coded_data$episodes` that identifies the case or dyad. Default is `"id"`.
- `time_limit` — Maximum episode-window length in seconds. `NULL` is allowed when `time_limit_frames` is supplied or when `constraint_method` is `"episode"`. Default is `3`.
- `time_limit_frames` — Optional maximum episode-window length in frames. If supplied, this takes precedence over `time_limit`.
- `constraint_method` — Character scalar controlling how the episode window ends: `"strict"`, `"episode"`, `"loose"`, or `"frames"`. Default is `"episode"`.
- `fps` — Frames per second used to convert `time_limit` to frames. Default is `30`.
- `missing_threshold` — Numeric scalar in `[0, 1]`. Denominator and numerator episodes are kept only when the comparison subject is present for at least this proportion of frames within the episode. Default is `0`.
- `exclude_emotions` — Character vector of emotions to exclude from the denominator calculation. Default is `"neutral"`.

Value

A data.table with columns `id`, `denominator`, `numerator`, `emotion`, `n_episodes`, and `synchrony`.

See Also

`synchrony_by_episode()`

Examples

```
library(data.table)

coding <- data.table(
  id = rep(1L, 6),
  subject = rep(c("teen", "parent"), each = 3),
  emotion = "happy",
  video_time = rep(1:3, 2),
  value = c(0.1, 0.2, 0.3, 0.1, 0.2, 0.3),
  in_state = c(FALSE, TRUE, FALSE, FALSE, FALSE, TRUE),
  run_id = c(1L, 1L, 1L, 2L, 2L, 2L)
)
episodes <- data.table(
```

```
  id = 1L,
  subject = c("teen", "parent"),
  emotion = "happy",
  run_id = c(1L, 2L),
  start_frame = c(2L, 3L),
  end_frame = c(2L, 3L)
)
coded_data <- structure(
  list(coding = coding, episodes = episodes),
  class = c("fr_coding", "list")
)
synchrony(coded_data, subject = "subject", id = "id", missing_threshold = 0)
```

---

synchrony_by_episode *Calculate synchrony by denominator episode*

Description

Calculate synchrony by denominator episode

Usage

```
synchrony_by_episode(
  coded_data,
  subject = "subject",
  id = "id",
  time_limit = 3,
  time_limit_frames = NULL,
  constraint_method = "episode",
  fps = 30L,
  missing_threshold = 0,
  exclude_emotions = "neutral"
)
```

Arguments

coded_data	Output from `convert_to_episodes()`.
subject	Character scalar giving the column in `coded_data$coding` and `coded_data$episodes` that identifies the subject. Default is `"subject"`.
id	Character scalar giving the column in `coded_data$coding` and `coded_data$episodes` that identifies the case or dyad. Default is `"id"`.
time_limit	Maximum episode-window length in seconds. `NULL` is allowed when `time_limit_frames` is supplied or when `constraint_method` is `"episode"`. Default is `3`.
time_limit_frames	Optional maximum episode-window length in frames. If supplied, this takes precedence over `time_limit`.
constraint_method	Character scalar controlling how the episode window ends: `"strict"`, `"episode"`, `"loose"`, or `"frames"`. Default is `"episode"`.
fps	Frames per second used to convert `time_limit` to frames. Default is `30`.

`missing_threshold`
: Numeric scalar in `[0, 1]`. Denominator and numerator episodes are kept only when the comparison subject is present for at least this proportion of frames within the episode. Default is `0`.

`exclude_emotions`
: Character vector of emotions to exclude from the denominator calculation. Default is `"neutral"`.

Value

A data.table with columns `id`, `denominator`, `numerator`, `emotion`, `run_id`, `start_frame`, `end_frame`, `present_prop`, and `synchrony`.

See Also

`synchrony()`

Examples

```
library(data.table)

coding <- data.table(
  id = rep(1L, 6),
  subject = rep(c("teen", "parent"), each = 3),
  emotion = "happy",
  video_time = rep(1:3, 2),
  value = c(0.1, 0.2, 0.3, 0.1, 0.2, 0.3),
  in_state = c(FALSE, TRUE, FALSE, FALSE, FALSE, TRUE),
  run_id = c(1L, 1L, 1L, 2L, 2L, 2L)
)
episodes <- data.table(
  id = 1L,
  subject = c("teen", "parent"),
  emotion = "happy",
  run_id = c(1L, 2L),
  start_frame = c(2L, 3L),
  end_frame = c(2L, 3L)
)
coded_data <- structure(
  list(coding = coding, episodes = episodes),
  class = c("fr_coding", "list")
)
synchrony_by_episode(coded_data, subject = "subject", id = "id", missing_threshold = 0)
```

`to_seconds` *Convert FaceReader time strings to seconds*

Description

Parses time values formatted as `hh:mm:ss` or `hh:mm:ss.s` and returns the equivalent number of seconds. Fractional seconds are truncated or padded to the number of digits specified by `digits`.

Usage

```
to_seconds(x, digits = 3L)
```

Arguments

`x`	Character vector of time strings.
`digits`	Number of fractional digits to retain after the decimal point. Set to `0` to discard fractional seconds. Default is `3`.

Value

A numeric vector of seconds.

Examples

```
## Not run:
to_seconds(c("00:00:10.000", "00:00:10.500"))

## End(Not run)
```

Index